

DIMAp - DIM0129 - 2026.2

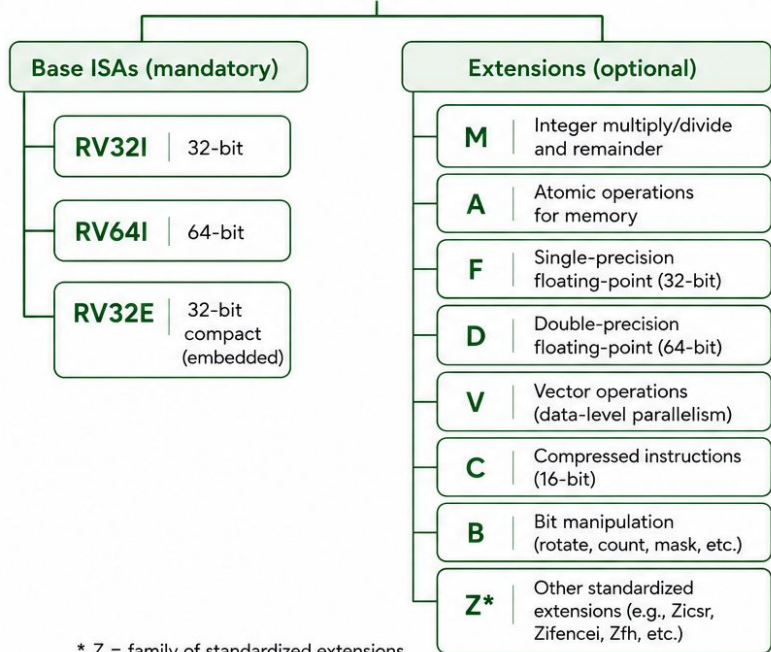
Hackeando o RISC-V: extensões da ISA

Monica Pereira

Extensões da ISA

RISC-V

modular ISA



* Z = family of standardized extensions.
(e.g., Zicsr, Zifencei, Zfh, etc.)



RISC-V: modular by design

Combines a minimal base ISA with extensions selected according to application needs.

Diferentemente de uma ISA que simplesmente acumula funcionalidades ao longo do tempo, o RISC-V foi concebido para permitir diferentes combinações de uma base com extensões. A especificação oficial define bases como RV32I/RV64I e extensões adicionais que podem ser combinadas.

Vantagem de extensões de ISA e não “evoluções” da ISA



Impacto no projeto

Adicionar mais recursos à ISA ou ao hardware tem custos.



+ hardware



+ área



+ potência



+ complexidade



+ validação



Necessidades diferentes

Processadores e aplicações não exigem os mesmos recursos.



Microcontrolador

não precisa do mesmo hardware de um servidor



Servidor

precisa de recursos que um microcontrolador talvez dispense



IA/HPC

pode precisar de vetores



Multicore

precisa de mecanismos de sincronização



Por isso, o RISC-V usa uma ISA modular: base + extensões.

Como uma nova instrução nasce
no software e chega ao hardware

Multiplicação

11.1. "M" Extension for Integer Multiplication and Division, Version 2.0

This chapter describes the standard integer multiplication and division instruction extension, which is named `M` and contains instructions that multiply or divide values held in two integer registers.

NOTE

We separate integer multiply and divide out from the base to simplify low-end implementations, or for applications where integer multiply and divide operations are either infrequent or better handled in attached accelerators.

<https://docs.riscv.org/reference/isa/v20260120/unpriv/m-st-ext.html>

RV32M Multiply Extension

31		25 24		20 19		15 14		12 11		7 6		0	
funct7				rs2		rs1		funct3		rd		opcode	
7				5		5		3		5		7	
Inst	Name	FMT	Opcode	funct3	funct7	Description (C)							
mul	MUL	R	0110011	0x0	0x01	rd = (rs1 * rs2)[31:0]							
mulh	MUL High	R	0110011	0x1	0x01	rd = (rs1 * rs2)[63:32]							
mulhsu	MUL High (S) (U)	R	0110011	0x2	0x01	rd = (rs1 * rs2)[63:32]							
mulhu	MUL High (U)	R	0110011	0x3	0x01	rd = (rs1 * rs2)[63:32]							
div	DIV	R	0110011	0x4	0x01	rd = rs1 / rs2							
divu	DIV (U)	R	0110011	0x5	0x01	rd = rs1 / rs2							
rem	Remainder	R	0110011	0x6	0x01	rd = rs1 % rs2							
remu	Remainder (U)	R	0110011	0x7	0x01	rd = rs1 % rs2							

Exemplo: Multiplicação no RV32I (RISC-V base de 32 bits)

Queremos calcular:

$$c = a * b;$$

Em um processador que implementa apenas **RV32I**, não existe uma instrução de multiplicação inteira.

Exemplo: Multiplicação no RV32I (RISC-V base de 32 bits)

A operação precisaria ser feita por software, por exemplo por uma rotina com somas, shifts etc.

```
int result = 0, b = 4, a = 10;

for (int i = 0; i < b; i++) {
    result = result + a;
}
```

```
addi t0, zero, 0      # result = 0
addi t3, zero, 0      # i = 0

LOOP:
    bge t3, t2, END    # if i >= b, finish

    add t0, t0, t1      # result = result + a
    addi t3, t3, 1      # i++

    jal zero, LOOP     # repeat

END:
```

Exemplo: Multiplicação no RV32I (RISC-V base de 32 bits)

Implementação melhor:

Por que?

```
addi t0, zero, 0      # result = 0

LOOP:
    beq  t2, zero, END # if b == 0, finish

    andi t3, t2, 1      # Check least significant bit of b
    beq  t3, zero, SKIP

    add  t0, t0, t1      # result += a

SKIP:
    slli t1, t1, 1      # a = a << 1
    srli t2, t2, 1      # b = b >> 1
    jal  zero, LOOP

END:
```

Como adicionar a multiplicação ao RISC-V

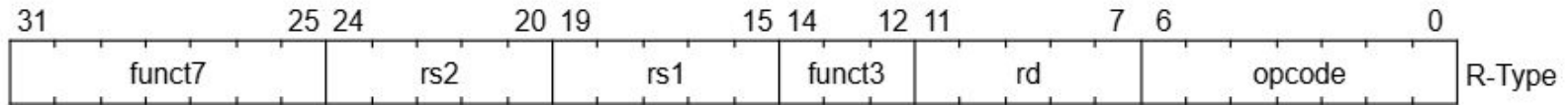
1. Definir a instrução seguindo o padrão assembly do RISC-V

Por ser RISC, operações na ULA usam somente registradores e imediatos

`mul t0, t1, t2`

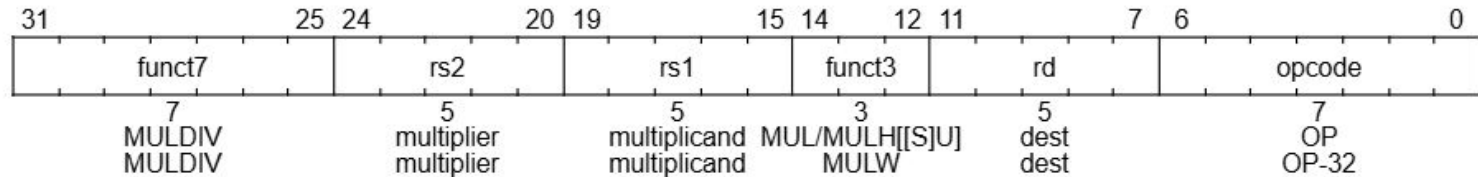
Como adicionar a multiplicação ao RISC-V

2. Resgatando o formato de instrução R (register)



Como adicionar a multiplicação ao RISC-V

3. Definindo os bits da instrução



A ISA define a instrução semântica:

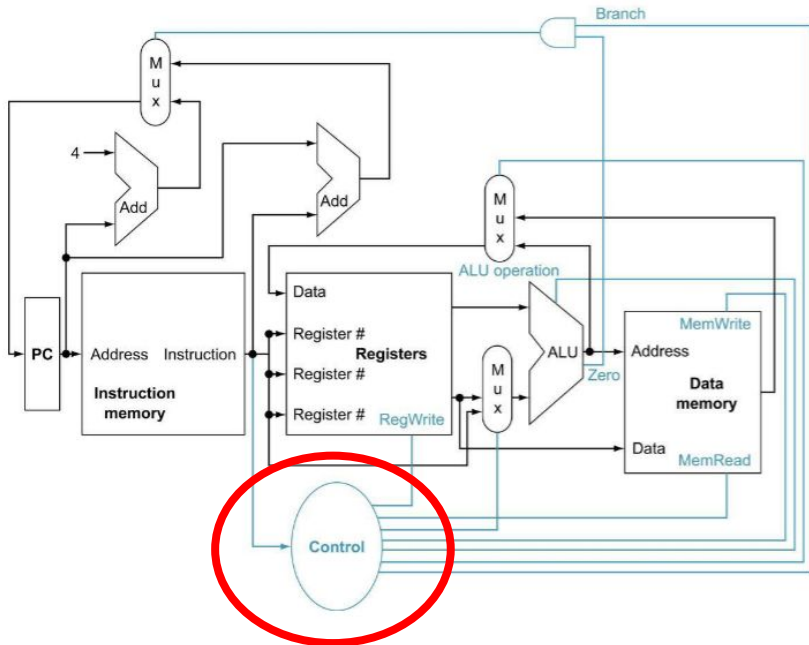
$$rd \leftarrow rs1 \times rs2$$

codificação:

opcode + funct3 + funct7

Como adicionar a multiplicação ao RISC-V

4. O decodificador precisa conhecer a instrução



Files

main

Go to file

doc

src

multi_cycle_processor

single_cycle_processor

ArithmeticLogicUnit.sv

ComparatorUnit.sv

CoreControlUnit.sv

DataMemory.sv

ImmediateSignExtend.sv

InstructionMemory.sv

LoadStoreUnit.sv

ProcessorCore.sv

RegisterFile.sv

risc_v_32_i_pkg.sv

tb

CODE_OF_CONDUCT.md

CONTRIBUTING.md

LICENSE

README.md

RV32I / src / single_cycle_processor / CoreControlUnit.sv

Code

Blame

371 lines (356 loc) · 15 KB

```
42
43     always_comb
44     begin
45
46         unique case(op_code_i)
47             OP_R_TYPE:
48                 begin
49                     pc_mux_sel_o = 1'b0;
50                     reg_write_enable_o = 1'b1;
51                     imm_select_o = IMM_UNKNOWN_TYPE;
52                     execute_port_a_sel_o = 1'b1;
53
54                     unique case({funct7_bit5_i, funct3_i})
55                         4'b0000:
56                             begin
57                                 alu_op_sel_o = OP_ADD;
58                                 comp_op_sel_o = OP_UNKNOWN;
59                                 execute_port_b_sel_o = 1'b1;
60                                 reg_write_data_sel_o = RD_MUX_ALU;
61                             end
62                         4'b1000:
63                             begin
64                                 alu_op_sel_o = OP_SUB;
65                                 comp_op_sel_o = OP_UNKNOWN;
66                                 execute_port_b_sel_o = 1'b1;
67                                 reg_write_data_sel_o = RD_MUX_ALU;
68                             end
69                         4'b0001:
70                             begin
71                                 alu_op_sel_o = OP_SLL;
72                                 comp_op_sel_o = OP_UNKNOWN;
73                                 execute_port_b_sel_o = 1'b1;
74                                 reg_write_data_sel_o = RD_MUX_ALU;
75                             end
76                     end
77                 end
78             default:
79                 begin
80                     pc_mux_sel_o = 1'b0;
81                     reg_write_enable_o = 1'b1;
82                     imm_select_o = IMM_UNKNOWN_TYPE;
83                     execute_port_a_sel_o = 1'b1;
84                 end
85             endcase
86         endcase
87     end
```

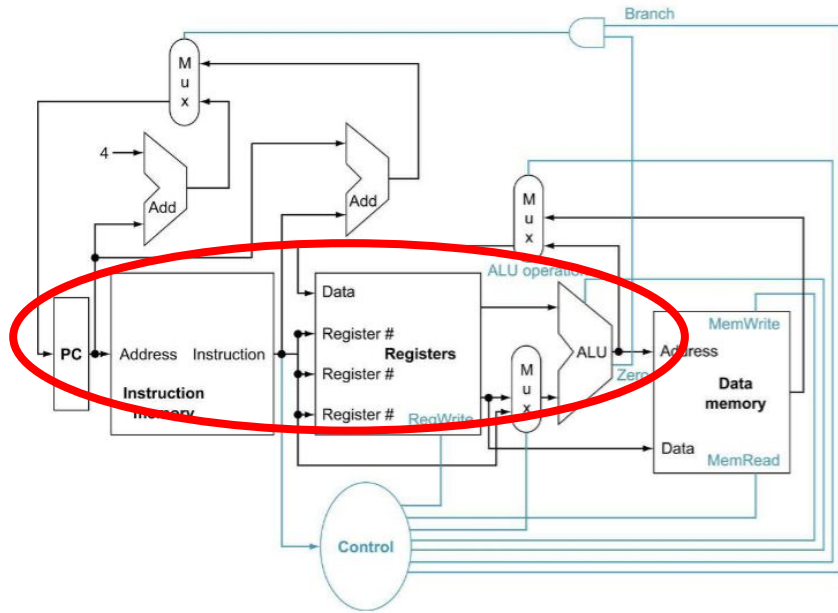
Code Blame 371 lines (356 loc) · 15 KB

```
42
43     always_comb
44     begin
45
46         unique case(op_code_i)
47             OP_R_TYPE:
48             begin
49                 pc_mux_sel_o = 1'b0;
50                 reg_write_enable_o = 1'b1;
51                 imm_select_o = IMM_UNKNOWN_TYPE;
52                 execute_port_a_sel_o = 1'b1;
53
54                 unique case({funct7_bit5_i,funct3_i})
55                     4'b0000:
56                     begin
57                         alu_op_sel_o = OP_ADD;
58                         comp_op_sel_o = OP_BUNKNOWN;
59                         execute_port_b_sel_o = 1'b1;
60                         reg_write_data_sel_o = RD_MUX_ALU;
61                     end
62                     4'b1000:
63                     begin
64                         alu_op_sel_o = OP_SUB;
65                         comp_op_sel_o = OP_BUNKNOWN;
66                         execute_port_b_sel_o = 1'b1;
67                         reg_write_data_sel_o = RD_MUX_ALU;
68                     end
```

```
4'b1000:
begin
    alu_op_sel_o = OP_MUL;
    comp_op_sel_o = OP_BUNKNOWN;
    execute_port_b_sel_o = 1'b1;
    reg_write_data_sel_o = RD_MUX_ALU;
end
```

Como adicionar a multiplicação ao RISC-V

5. O datapath precisa executá-la



Files

- main
- doc
- src
 - multi_cycle_processor
 - single_cycle_processor
 - ArithmeticLogicUnit.v
 - ComparatorUnit.v
 - CoreControlUnit.v
 - DataMemory.v
 - ImmediateSignExtend.v
 - InstructionMemory.v
 - LoadStoreUnit.v
 - ProcessorCore.v
 - ProcessorFile.v
 - risc_v_32_i_pkg.v
- tb
 - CODE_OF_CONDUCT.md
 - CONTRIBUTING.md
 - LICENSE
 - README.md

RV32I / src / single_cycle_processor / ArithmeticLogicUnit.v

Code Blame 38 lines (35 loc) · 1.56 KB

```
3  * This code is licensed under MIT license (see LICENSE.txt for details)
4  *
5  * Module: ArithmeticLogicUnit.v
6  *
7  * Description:
8  *
9  * This does computation over the data related to arithmetic and logic
10 * instructions. This does not holds the comparison which is done by
11 * ComparatorUnit.
12 *****
13 import risc_v_32_i_pkg::*;
14
15 module ArithmeticLogicUnit #(parameter XLEN = 32, parameter REG_ADDR_WIDTH = 5)
16 (
17     output logic [XLEN-1:0] alu_o,
18     input logic [XLEN-1:0] alu_port_a_i,
19     input logic [XLEN-1:0] alu_port_b_i,
20     input alu_select_e      alu_op_sel_i
21 );
22
23 always_comb
24 begin
25     unique case(alu_op_sel_i)
26         OP_ADD:    alu_o = alu_port_a_i + alu_port_b_i;
27         OP_SUB:    alu_o = alu_port_a_i - alu_port_b_i;
28         OP_AND:    alu_o = alu_port_a_i & alu_port_b_i;
29         OP_OR:     alu_o = alu_port_a_i | alu_port_b_i;
30         OP_XOR:    alu_o = alu_port_a_i ^ alu_port_b_i;
31         OP_SLL:    alu_o = alu_port_a_i << alu_port_b_i[REG_ADDR_WIDTH-1:0];
32         OP_SRL:    alu_o = alu_port_a_i >> alu_port_b_i[REG_ADDR_WIDTH-1:0];
33         OP_SRA:    alu_o = $signed(alu_port_a_i) >>> alu_port_b_i[REG_ADDR_WIDTH-1:0];
34         OP_UNKNOMN: alu_o = {XLEN{1'b0}};
35     endcase
36 end
37
38 endmodule
```

Code Blame 38 lines (35 loc) · 1.56 KB

```

3  * This code is licensed under MIT license (see LICENSE.txt for details)
4  *
5  * Module: ArithmeticLogicUnit.sv
6  *
7  * Description:
8  *
9  * This does computation over the data related to arithmetic and logic
10 * instructions. This does not holds the comparison which is done by
11 * ComparatorUnit.
12 *****/
13 import risc_v_32_i_pkg::*;
14
15 module ArithmeticLogicUnit #(parameter XLEN = 32, parameter REG_ADDR_WIDTH =
16 (
17     output logic      [XLEN-1:0] alu_o,
18     input logic       [XLEN-1:0] alu_port_a_i,
19     input logic       [XLEN-1:0] alu_port_b_i,
20     input alu_select_e      alu_op_sel_i
21 );
22
23     always_comb
24     begin
25         unique case(alu_op_sel_i)
26             OP_ADD:    alu_o =      alu_port_a_i +  alu_port_b_i;
27             OP_SUB:    alu_o =      alu_port_a_i -  alu_port_b_i;
28             OP_AND:    alu_o =      alu_port_a_i &  alu_port_b_i;
29             OP_OR:     alu_o =      alu_port_a_i |  alu_port_b_i;
30             OP_XOR:    alu_o =      alu_port_a_i ^  alu_port_b_i;
31             OP_SLL:    alu_o =      alu_port_a_i << alu_port_b_i[REG_ADDR_WIDTH-1:0];
32             OP_SRL:    alu_o =      alu_port_a_i >> alu_port_b_i[REG_ADDR_WIDTH-1:0];
33             OP_SRA:    alu_o = $signed(alu_port_a_i) >>> alu_port_b_i[REG_ADDR_WIDTH-1:0];
34             OP_UNKNOWN: alu_o = {XLEN{1'b0}};
35         endcase
36     end
37
38 endmodule

```

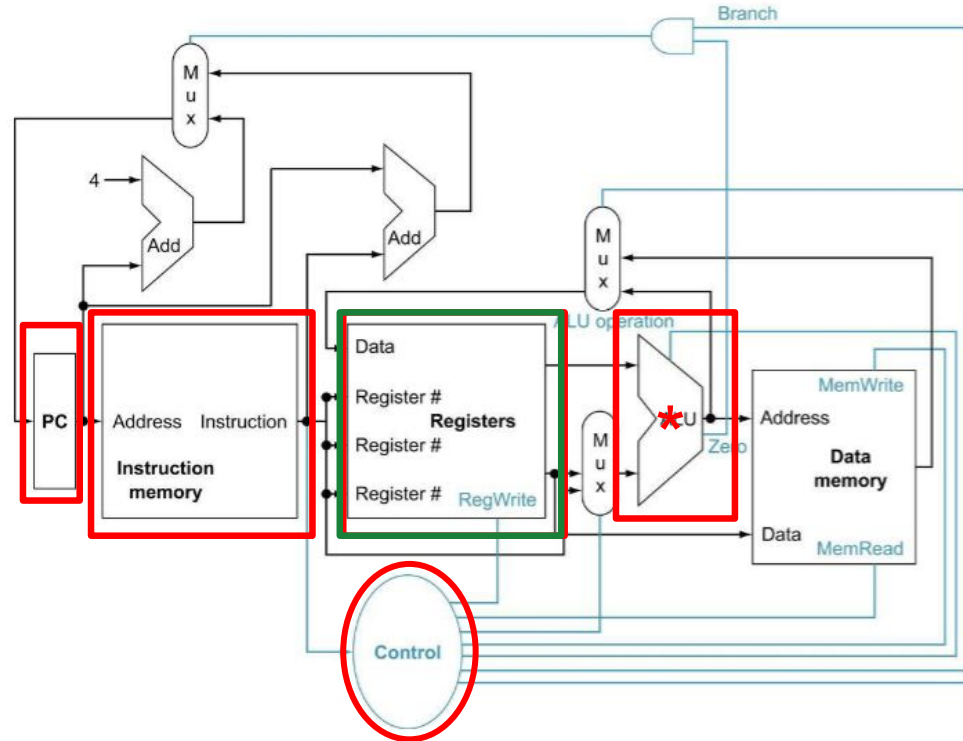
```

19     input logic      [XLEN-1:0] alu_port_b_i,
20     input alu_select_e      alu_op_sel_i
21 );
22
23     always_comb
24     begin
25         unique case(alu_op_sel_i)
26             OP_ADD:    alu_o =      alu_port_a_i +  alu_port_b_i;
27             OP_SUB:    alu_o =      alu_port_a_i -  alu_port_b_i;
28             OP_AND:    alu_o =      alu_port_a_i &  alu_port_b_i;
29             OP_OR:     alu_o =      alu_port_a_i |  alu_port_b_i;
30             OP_XOR:    alu_o =      alu_port_a_i ^  alu_port_b_i;
31             OP_SLL:    alu_o =      alu_port_a_i << alu_port_b_i[REG_ADDR_WIDTH-1:0];
32             OP_SRL:    alu_o =      alu_port_a_i >> alu_port_b_i[REG_ADDR_WIDTH-1:0];
33             OP_SRA:    alu_o = $signed(alu_port_a_i) >>> alu_port_b_i[REG_ADDR_WIDTH-1:0];
34             OP_MUL:    alu_o =      alu_port_a_i *  alu_port_b_i;
35             OP_UNKNOWN: alu_o = {XLEN{1'b0}};
36         endcase
37     end
38
39 endmodule

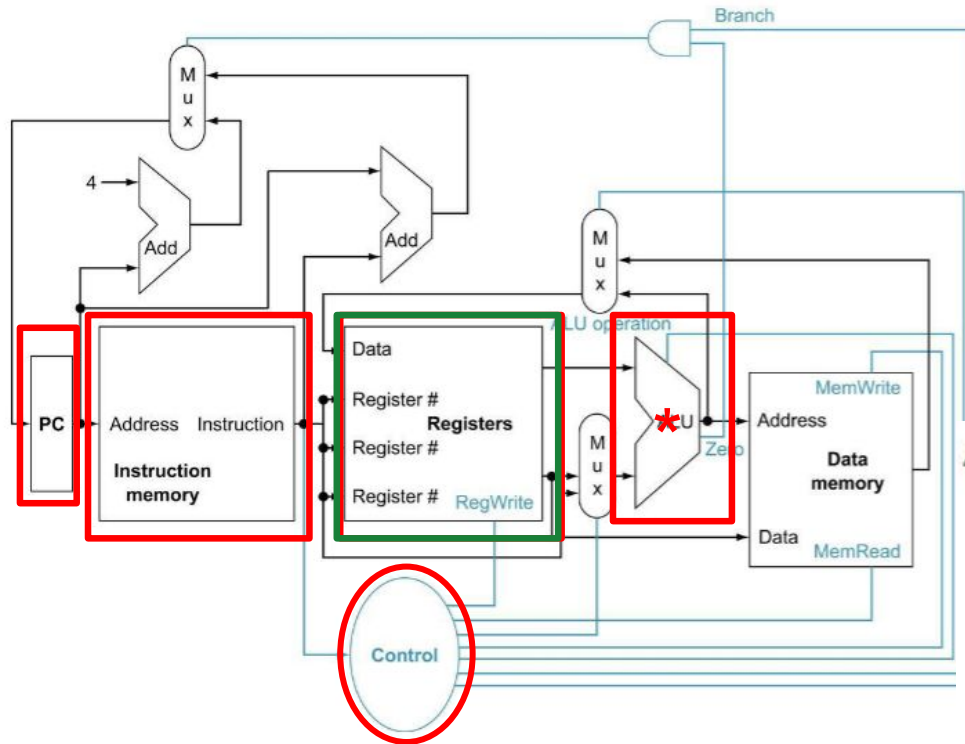
```

Como adicionar a multiplicação ao RISC-V

6. O controle precisa selecionar o novo caminho



Como adicionar a multiplicação a



```

module CoreControlUnit #(parameter XLEN = 32)
(
    output logic                pc_mux_sel_o,
    output logic                reg_write_enable_o,
    output imm_select_e         imm_select_o,
    output logic                execute_port_a_sel_o,
    output logic                execute_port_b_sel_o,
    output alu_select_e         alu_op_sel_o,
    output comp_select_e        comp_op_sel_o,
    output load_store_type_e    load_store_type_o,
    output logic                data_memory_write_enable_o,
    output write_data_select_e  reg_write_data_sel_o,
    input logic [6:0]           op_code_i,
    input logic [2:0]           funct3_i,
    input logic                 funct7_bit5_i,
    input logic                 branch_enable_i
);

4'b1000:
begin
    alu_op_sel_o = OP_MUL;
    comp_op_sel_o = OP_BUNKNOWN;
    execute_port_b_sel_o = 1'b1;
    reg_write_data_sel_o = RD_MUX_ALU;
end
    
```

V Extension (vector)

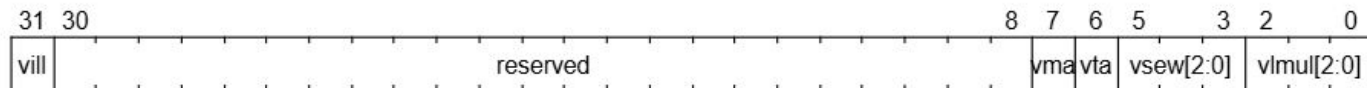
30.1. "V" Standard Extension for Vector Operations, Version 1.0

NOTE

The base vector extension is intended to provide general support for data-parallel execution within the 32-bit instruction encoding space, with later vector extensions supporting richer functionality for certain domains.

<https://docs.riscv.org/reference/isa/v20260120/unpriv/v-st-ext.html>

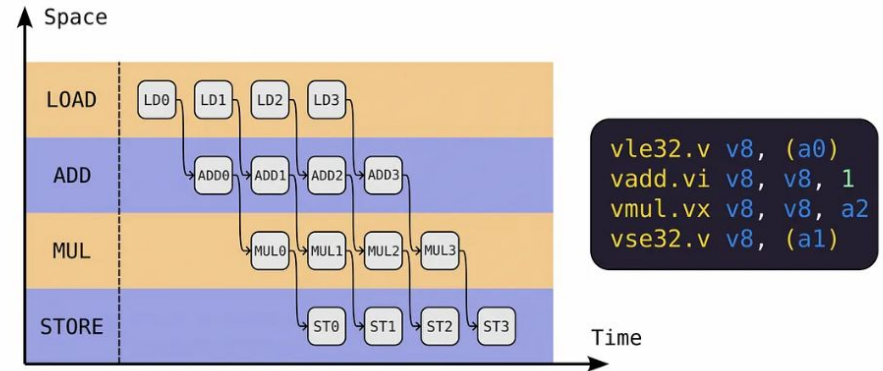
The `vtype` register has five fields, `vill`, `vma`, `vta`, `vsew[2:0]`, and `vlmul[2:0]`. Bits `vtype[XLEN-2:8]` should be written with zero, and non-zero values in this field are reserved.



Vector Processor

The idea of a vector processor is based on having a few basic functional blocks in a CPU and chaining them to process elements of a vector in parallel. Each functional block does one operation with one element of a vector in a single unit of time. Because of the data independence the functional blocks can work in parallel, processing different elements of the vector. The chaining is not dissimilar to the pipelining in the superscalar CPUs (however there is a significant difference: pipelining is based on instruction parallelism and vector chaining on the data parallelism).

In the example code on the picture the processor loads the vector from memory, adds a constant 1 to each element, multiplies each element by 2 and stores the vector back to the memory. In this sequence of operation in the first clock cycle the LOAD unit will load a first element from the vector to the vector register. In the next cycle the LOAD unit will load a second element and the ADD unit will simultaneously add a constant to the first element of the vector.

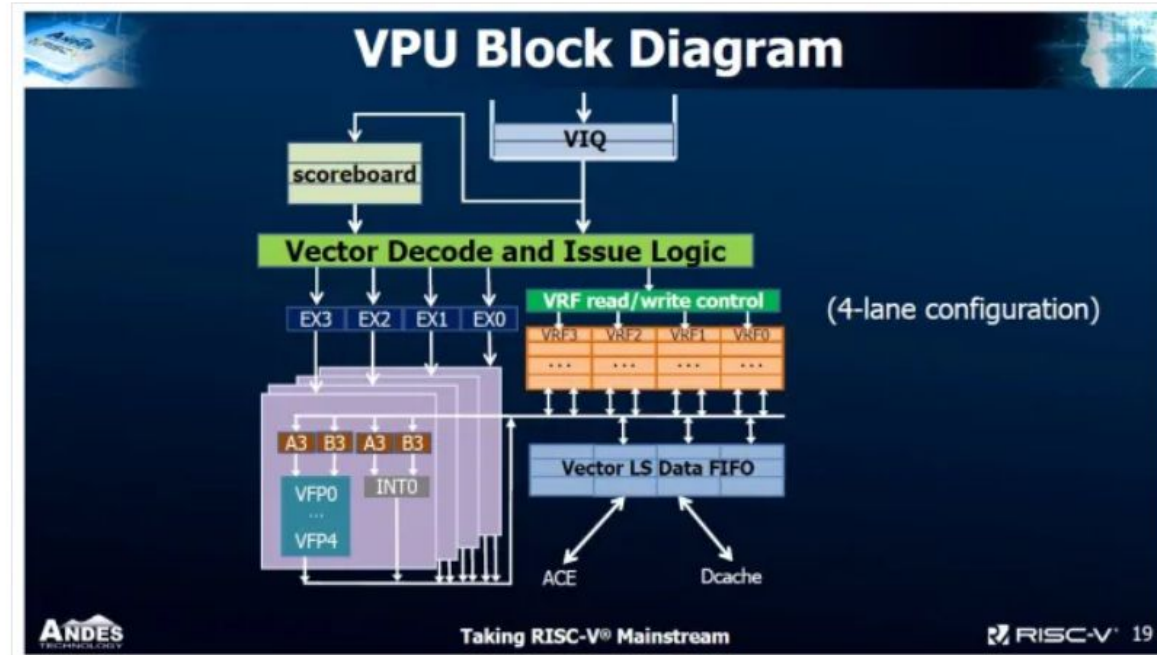


A diagram of a vector processor's functional blocks operations. The executed RISC-V assembly code is shown on the right. Notice the chaining — while the first element moves down on the chain of functional blocks the first functional block can start processing of the next vector element.

Principais mudanças na Extensão

1. Banco de registradores vetoriais adicional (tamanho definido por VLEN)
2. Registrador VL (Vector Length): informa quantos elementos serão processados pela próxima operação vetorial
3. Definição da quantidade de bits de cada elemento (SEW — Selected Element Width)
4. Instruções para configurar o estado vetorial (instrução vsetvli que indica VL, SEW, LMUL)
5. Instruções vetoriais
 - a. `vadd.vv v3, v1, v2` $v3[i] = v1[i] + v2[i]$
6. Memória vetorial: interface entre banco de registradores vetorial e memória é modificada
7. Uso de máscaras: alterar apenas elementos que satisfazem uma condição

Andes' Core has RISC-V Vector Instruction Extension



The NX27V contains a vector processing unit (VPU) which supports the RVV scalable vector instruction set, designed from the ground up to be a Cray-like full vectorization computation unit (Image: Andes Technology)

Generalizando a criação de uma nova
extensão ou instrução

Roteiro para criação de novas instruções

1. Definir semântica
2. Definir codificação
3. Modificar decodificador
4. Modificar controle
5. Adicionar ou modificar o caminho de dados
6. Validar
7. Avaliar métricas

Projeto da Unidade 2

Descrição do projeto

- Criar três novas instruções da tabela a seguir e adicionar ao RV32I (RISC-V base)
 - Propor o formato das instruções
 - Propor mudanças no controle, decodificação e caminho de dados
 - Implementar no RISC-V Verilog
- Os projetos RISC-V com as instruções novas devem ser validados através de simulação
- PRAZO: 21/10

ATENÇÃO: Grupo de até 3 integrantes e cada grupo deve implementar as 3 instruções!!!

Instruções

Instrução proposta	Semântica	Relação com IA
<code>max rd, rs1, rs2</code>	<code>rd = max(rs1,rs2)</code>	Pooling / redução
<code>mac rd, rs1, rs2</code>	<code>rd = rd + rs1*rs2</code>	Redes neurais, convolução
<code>sign rd, rs1</code>	$rd = \begin{cases} -1, & rs1 < 0 \\ 0, & rs1 = 0 \\ 1, & rs1 > 0 \end{cases}$	Ativação/ classificação/ quantização

<https://github.com/HarieshAnbalagan/RV32I>